

Standardized Analysis of Spatial Omics Data

Since the introduction of spatial transcriptomics nearly 10 years ago, the amount of data in the field of spatial omics has exploded but without much in terms of a standardized approach to analysis of this data. There are now multiple platforms for spatial transcriptomics, multiple types of omics (transcriptomics, proteomics, etc.), all with varying advantages and disadvantages that can make it difficult for researchers to plan their experiments. Here, we try to lay out a few approaches to standardizing different steps of the process to allow for scientists to have better control over their experiments as well as promoting easy and open sharing of data and analysis pipelines.

Topics

1. Introduction

Introduction

For the sake of clarity, we will use a mouse model as our point of discussion, but the same basic tenets hold true for any model species, including humans. Different platforms use different cellular resolutions and names for our points of interest (probes, spots, cells, etc.), which we will call probes for simplicity. In a perfect world, we would like to be able to compare data across various platforms and experimental conditions, and this can only be done with an eye turned towards standardization. To do this, we must consider a few things, including the nature of working with data on computers.

Computers work on fairly strict rules and logic, and while many operating systems have built in methods to account for variations in naming, it is good practice to be consistent with our naming and structuring so as to prevent any possible confusion. This includes avoiding using spaces within names of files. Most operating systems and tools are now capable of accounting for spaces in names, but since much of what we do will involve using computational tools that may or may not be based around the command line, we should try to avoid this common source of command line errors. This means that rather than using a name such as ‘subject 001’ we would preferentially use a name such as ‘subject-001’. Using a delimiter such as a hyphens or underscores in this way we can easily separate unique portions of our filenames without using spaces, while also being command line friendly.

Components of spatial omics data

When working with spatial omics data, each format can have its own peculiarities, but there are certain types of data that we can expect from each dataset. These include: histological images, gene/protein expression matrices, lists of coordinates for probes, barcodes or other identifier for a file containing information about transformations between image space and real world space and other information.

This is not to say that additional information contained in each sample is not important, but that it is not necessarily universal and might not be as easy to standardize across platforms. These data may come in different formats from different platforms, but their contents should be analogous. For example, a gene expression matrix may come as a .csv file or as part of a .h5 file, but the concept of having a matrix where either the columns or rows represent a gene, and the other direction represents the probe is the same for both.

Data structure for processing

In order to allow researchers to better interact with each other and share data freely, we must have some sense of unified structure to how we store and share our data. One useful example for this can be seen in the neuroimaging community in the form of BIDS. There are many details to the data structure implemented in BIDS, but one important component is that data should be organized in a repeatable and reusable format. This requires researchers to plan ahead when creating experiments, and includes the common stumbling points of proper subject identification and file naming. Generally helpful aspects of BIDS include the creation of four distinct folders to be used throughout our experiment. These include ‘rawdata’, ‘derivatives’, ‘code’, and ‘sourcedata’. The BIDS specification provides a very detailed and clear description of these folders, but in short we have three folders that contain our data, and one folder that contains the code used in processing that data. The ‘sourcedata’ folder is where we store the original data as downloaded from whatever machine our experiment was collected on. This could be stored as .zip files, .tiff files, folders, or many different sorts. This data will be used to generate our ‘rawdata’, which is data formatted in a way that it is easy to access during our computations. Data in the ‘rawdata’ folder should be stored inside of ‘participant’ folders and optionally ‘session’ folders within the participant folder. An example of a session in spatial omics data might be that you have two slices of tissue from one subject that have undergone different treatments.

Subject IDs

It can be easy, especially for an early career researcher, to use subject identification numbers that are too short, ambiguous, or repeated across different experiments. This problem is amplified when those subject IDs become identifiers used in computational analysis. A human might be able to read a poorly assigned subject ID and know what was intended—partly because they are usually the one assigning the ID—but a computer cannot so easily make this distinction. The first useful consideration comes in the form of leading zeroes. When you create an identification number, you should plan ahead that you will likely have dozens, hundreds, or more animals throughout your experiments and career. Therefore, when naming a subject ‘subject-1’ you should instead consider using multiple leading zeroes, such as ‘subject-001’ or ‘subject-0001’. By using three digits (i.e. ‘000’) we have given ourselves enough digits to account for 1000 total animals

(000-999), and by using four digits we can now account for up to 10,000 total subjects (0000-9999). All of the data for a single subject should be contained within a folder using the name of the subject.

Let us next consider the importance of experimental distinction. If you have three sets of experimental animals and all three of them include ‘subject-001’ through ‘subject-010’, there is no obvious way for a computer or future researcher to be able to distinguish between the various ‘subject-1’ animals. This can be handled with multiple safeguards.

Session IDs

While seemingly less frequent (so far!) in spatial omics data than in neuroimaging data, it is very likely an experiment might utilize data from the same subject for multiple tissue collections. If this is the case, it is good practice to use session IDs that are informative. This can be something as simple as a date, in the case that the samples were collected at different time points. If this is the case, a good approach is to use the format of yyyyMMDD, or some variation thereof, as this allows for numerical sorting by computers, i.e. you can sort sessions in order of date collected by sorting alphanumerically. This would not be the case if using either MMDDyyyy or DDMMyyyy formats.

Participants list

Another useful tool to be gleaned from the BIDS format is the usage of a file called ‘participants.tsv’ that contains information about all subjects within a single experiment. This file can be as sparse as just a list of the subject IDs, or highly detailed including information such as sex and genotype of each subject, as well as the date the tissue was collected.

Analysis pipelines

Code specific to the processing and analysis of your current experiment should be stored within the ‘code’ folder. These pipelines typically include, but are not limited to: preprocessing the data,

Writing clean code

There are detailed writings about how to write clean and compact code, but those are typically reserved for computer scientists. As computational biologists, we should try to take what we can from these guidelines without having to become computer scientists as well. We will try to outline some of the key elements of writing good code without burdening the reader too much. A major theme to keep in mind is that while our code will be run by a computer, it will be heavily read by humans, and so should be clear and informative to anyone who reads it after us, which includes using good variable names, descriptive comments, and structuring our code in a readable format. This will not be a

guide of how to code, but rather how to format code in a useful way for sharing with other scientists. Examples will be predominantly given in Python, but the concepts should be generally applicable.

Variables

Variable names should be readable and informative. Variable names also cannot begin with a number, cannot contain special characters (i.e. !, @, #, etc.) or spaces. When creating variable names, it is recommended to use either snake case ('snake_case') or camel case ('camelCase') in order to make the names more readable. Generally it is recommended to choose one of these styles and stick to it throughout your code.

Functions

Functions should be written for tasks that are regularly repeated in order to avoid using extra lines of code on repetitive tasks that might otherwise involve copying and pasting lines of code over and over within a script. It is good practice to include a description of how to use the function as part of a 'docstring' within the code. This description should include: a brief description of what the function does, any input variables the function expects and their description, and what the function returns (if anything) as an output of running.

Programming terminology

While this isn't a lesson in programming, it might be helpful to provide some terminology for context and clarification.

script - a file containing code that will be run linearly to perform a certain task, such as analyzing a spatial omics dataset. Is not installed but run directly, either from the command line or within an interactive development environment (IDE)

program - a compiled set of code that is usually installed onto a machine can be used to perform

interactive development environment (IDE) - a program that is used for the writing of code. Often something like VSCode, Spyder, Matlab, etc.